

HostNet - the guest's sockets, served by the host

HostNet: the guest's sockets, served by the host

How the RISC OS Pi 4 emulator took networking out of the emulated machine. RISC OS programs still make the same socket calls, but a small module hands each one to the emulator, and the emulator makes it on the host. The guest has no IP stack, no network card, no DHCP and no link layer.

RISC OS used to reach the network the way a real Raspberry Pi does: its own TCP/IP stack in ROM, a network driver, and an emulated USB network card on QEMU's user-mode network. That path worked above the link layer and failed below it. DHCP obtained an address in none of twelve measured boots, and the emulated card needed 24 USB bulk transactions to move one 1500-byte frame.

HostNet replaces the lot at the socket API. A RISC OS module that calls itself **Internet 6.00** claims the same 35 socket SWIs as the Internet module. Each SWI copies its registers into a request block and rings a doorbell. A QEMU device at the address of the Pi 4's unmodelled Ethernet controller answers by making real, non-blocking socket calls on the host. HostFS already works the same way for files.

The design and Sprints 0 to 3 all landed on 14 September 2026. Two follow-ups hardened the switch-off and start-up behaviour that evening, and the Windows release shipped HostNet, switched on, on the 15th.

This document describes the fork's `riscos-pi4` branch as published at `d535505fad`, 15 September 2026, and names commits by hash. Some design background paraphrases the fork's private design record; everything else comes from the public code and commit messages. The [HostFS walkthrough](#) covers the first doorbell, whose transport HostNet copies.

These documents

Chapter	What it covers
1. The network the guest used to have	the ROM stack over an emulated USB card, and what was measured to fail
2. Where to cut	the socket SWI level, the module's contract, the sprints and what changed as they landed
3. A doorbell in the GENET window	the device, its registers, the request block, and switching it off
4. Never blocking the host	guest memory, the retry answer, connect, readiness and Select
5. The verbs	the 35 SWIs, and translating addresses, flags, options and errors

These documents, continued

Chapter	What it covers
6. Internet Event 19 without an interrupt	the ticker, the callback, the wake-up poll, and reasons that change
7. Internet 6.00 in 750 lines	the guest module: building it, finding the doorbell, waiting, refusing to load
8. Shipping it	the developer switch, the Windows release's module folder, the menu and the installer
9. Proven, and what comes next	the tests, the numbers, the stated limits, and work in progress

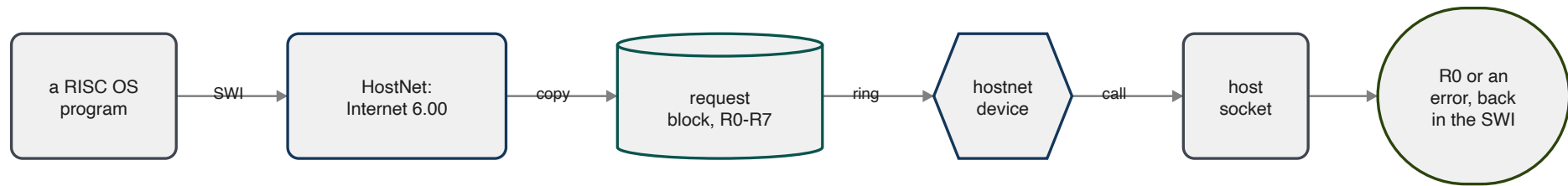
At a glance

Guest module	Internet 6.00, SWI chunk &41200, the Internet module's 35 socket SWIs in the same order
Doorbell	hostnet device at 0xFD580000, the BCM2711 GENET window; magic 'HNET', one 64-byte request block per call
Host side	real host sockets, always non-blocking; a would-block becomes HN_RC_RETRY and the guest waits
Events	a 50 Hz ticker asks the host which sockets woke, and the module raises Internet Event 19

At a glance, continued

Measured	NetSurf loaded riscosopen.org over TLS in 1.3 s; the old stack could not resolve the name
Switch	-global hostnet.sockets=on; off, the whole window reads zero as before
Shipped	Windows release RISCOSQEA72v7, switched on, as Modules\HostNet, ffa on the disc
Code	hw/misc/hostnet.c, include/hw/misc/hostnet.h, riscos-pi4/hostnet/, ui/dx11.c, the release scripts

One socket call, from a program to the host



Reading the diagrams

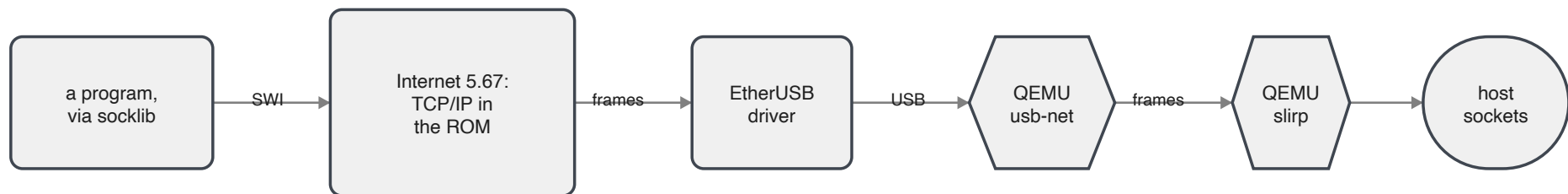
Every diagram colours its outlines by what each box is:

Outline	Meaning
dark blue	code in this fork: the device, the module, the launchers and scripts
grey	code from elsewhere: RISC OS, QEMU, the host operating system
purple	a file or data artefact: a module file, a setting
teal	state at run time: a request block, a table, guest memory
amber	a decision or a test
green	the outcome

1. The network the guest used to have

Before HostNet, the emulated machine did its networking exactly as a Raspberry Pi 4 running RISC OS does, only with an emulated network card. This chapter describes that path, the socket interface RISC OS programs use, and the measurements that showed where it failed. Nothing here criticises the RISC OS stack itself. Above the link layer it worked; the trouble was everything below it.

The old path



Every box in that chain is someone else's working code, which is why the diagram outlines them all in grey. A program built with the TCP/IP libraries makes a socket call. The ROM's Internet module turns it into TCP and IP packets and Ethernet frames. The EtherUSB driver sends those frames over USB to QEMU's emulated USB network device, `usb-net`. QEMU's user-mode network, `slirp`, then unwraps the frames and makes host socket calls to carry the data onwards.

The modules in the ROM

Numbered by position in the 5.30 ROM, which is how a CMOS setting unplugs them:

Chain	Module	Role in the old path
97	MbufManager	the memory buffers the stack and drivers share
98	Internet 5.67	the TCP/IP stack; provides the socket SWIs
99	Resolver	name lookups, over UDP sockets
106	EtherGENET	the Pi 4's own Ethernet driver; already unplugged, as no MAC is modelled
107	EtherUSB	the driver actually used, for the emulated USB card
108	DHCP	the address-configuration client

The emulated network

The launchers attached the card with `-netdev user` and `-device usb-net`, on port 3 of the emulated USB hub. slirp's network is fixed:

Address	What it is
10.0.2.0/24	the whole network
10.0.2.15	the guest
10.0.2.2	the host, which is also the gateway
10.0.2.3	slirp's DNS forwarder, reachable only from inside slirp

The socket interface, as RISC OS programs see it

RISC OS programs do not call a socket library in the operating system. They are built against the TCP/IP libraries, whose `socklib` turns each C call into a SWI in the Internet module's chunk, starting at &41200: `Socket_Creat`, `Socket_Connect`, `Socket_Recv` and so on.

Three facts that shaped HostNet

Fact	What it means
the registers are the C arguments	the Internet module treats the register block as the BSD call's arguments; the result comes back in R0
an error is a number	V set, with an error block numbered &20E00 plus the errno; <code>socklib</code> subtracts &20E00 and ignores the text
there are two ABI generations	six SWIs have an Internet 4 form, with a 16-bit family field in the address, and a 4.4BSD _1 form; both are still used

A module that copied R0 to R7 somewhere, had the call made, and put the result back in R0 would therefore serve every existing program without the program knowing. That is the idea chapter 2 builds on.

What failed, measured

The Sprint 0 commit, `23fc9630c5`, gives the reason for the whole project in one paragraph: measured over twelve boots, the stack above the link layer was fine, and the link layer was not. Its conclusion is short: "The host has a network stack already."

What failed

Finding	Number	Source
DHCP obtained an address	0 in 12 boots; the exchange itself took 2.5 s	23fc9630c5
USB packets	full speed only, 64-byte bulk packets, one 2 KB buffer	dev-network.c
one 1500-byte frame	24 USB bulk transactions	23fc9630c5

What was fixed, and what worked

Finding	Number	Source
failed USB control requests per boot	6, then none	75980f8cf8
desktop time, DHCP against a static address	49 s against 28 s	d0de1c013c
above the link layer	ARP, TLS and six parallel HTTPS connections worked	23fc9630c5

The link layer, in the source

The card's descriptors explain the 24 transactions: 64-byte bulk packets, a full-speed descriptor only, and one 2 KB receive buffer, so a 1500-byte frame is 24 packets, each its own transaction, with no pipelining. From QEMU's `hw/usb/dev-network.c`:

```
        .bEndpointAddress    = USB_DIR_IN | 0x02,
        .bmAttributes         = USB_ENDPOINT_XFER_BULK,
        .wMaxPacketSize       = 0x40,
    ...
    .full = &desc_device_net,
    ...
    unsigned int in_ptr, in_len;
    uint8_t in_buf[2048];
    ...
    /* Only accept packet if input buffer is empty */
    if (s->in_len > 0) {
        return 0;
    }
```

Two fixes that came first

Two commits earlier on 14 September tried to make the old path behave. Name lookups still failed intermittently afterwards: when Sprint 2 was measured, NetSurf on the old stack could not resolve a host name at all. What the two commits found made HostNet the next step:

Commit	What it changed	What it found
75980f8cf8	usb-net answers the endpoint status requests EtherUSB makes, six per boot; slirp gains a domain name	DHCP still fails: slirp answers within the millisecond, and the guest discards the offer and falls back to a link-local address
d0de1c013c	the release disc sets the address, gateway and OpenDNS name servers directly	the desktop in 28 seconds instead of 49, with no DHCP on the wire

2. Where to cut

Taking networking out of the guest means choosing a boundary: everything above it stays in RISC OS, and everything below it moves to the host. This chapter explains why the boundary is the socket SWIs, lists the contract the guest module had to keep, and follows the plan through the four sprints that landed on 14 September, including what changed on the way.

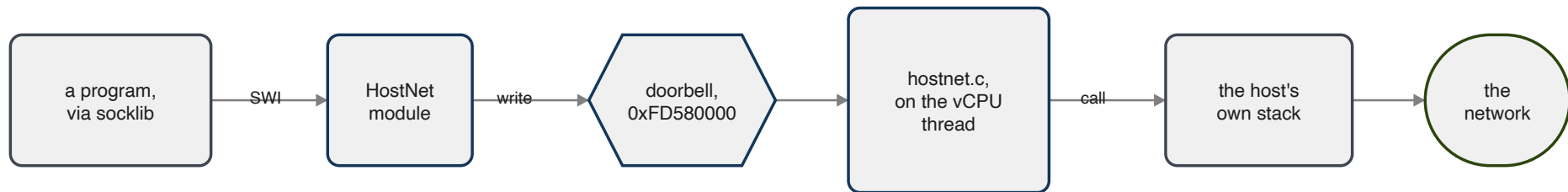
Three places to cut

The candidates

Cut at	What moves to the host	Why not, or why
the link layer: a better emulated card	only the card	the guest still runs DHCP, ARP and IP over an emulated device, which is where the failures were
the library: a new <code>socklib</code>	the socket calls of programs rebuilt against it	every existing program would need rebuilding, and most cannot be
the socket SWIs	everything below the program	every prebuilt program works unchanged, because the SWIs are the interface they already use

The SWI level wins because of the interface facts in chapter 1. Each socket SWI's registers are the BSD call's arguments, and its result is R0 or an error numbered from &20E00. A module that forwards R0 to R7 and returns R0 needs no knowledge of what any call means. The guest module's own header comment says that this contract "is why this module can be this small".

The path after the cut



Compared with the old path, the Internet module, EtherUSB, DHCP, the emulated card and slirp have all gone from the route a socket call takes. Resolver stays: it does name lookups over UDP sockets, which HostNet now serves. TLS stays in the guest too. The Sprint 2 commit says it this way: "HostNet carries bytes, not certificates."

The contract

The module is small, but it has to satisfy everything on the disc that expects the Internet module. Each line of this contract is recorded in the code or in a sprint commit:

What the module promises

Promise	Why
Title Internet, version 6.00	the boot sequence is full of <code>RMEnsure Internet 5.40</code> and <code>RMEnsure Internet 5.02</code> ; 6.00 satisfies them all without an edit
SWI chunk &41200, the same 35 SWIs in the same order	programs call SWIs by number; the list is generated from one file so the order cannot drift
The host never blocks	the doorbell is answered inside the guest CPU's memory write, holding QEMU's global lock
Resolver and TLS stay in the guest	both already work over sockets, so they need nothing from the host but sockets

What the module promises, continued

Promise	Why
The transport copies HostFS's	a 64-byte header with R0 to R7 and guest logical addresses, reached by the same MMU walk; its own device, state and sequence numbers
IPv4 only	<code>AF_INET6</code> answers <code>EAFNOSUPPORT</code> ; IPv6 is in no sprint
Snapshots drop sockets	a host socket cannot be saved in an image, so a restored machine's descriptors answer <code>EBADF</code>
The guest is an application to the host	its sockets are the host user's own sockets

The name is the unusual part. Claiming another module's title and SWI chunk is normally a mistake. Here it is the plan: the ROM's own Internet 5.67 is either unplugged in CMOS or killed as HostNet loads, and every `RMEnsure Internet` check on the disc sees a newer stack and proceeds. Chapter 7 shows why the same choice makes it essential that the module refuses to load where there is no doorbell.

Six sprints, four landed

The work was planned as six sprints. The first four each have a public commit, and each commit message gives the test that closed it:

The sprints

Sprint	Goal	Status at d535505fad
0	the transport; every call "not supported"	23fc9630c5, 19:09
1	UDP, and a name resolved over it	0f73da70af, 19:52
2	TCP, and NetSurf on the real web	89013ef0f2, 20:28
3	listeners, message calls, event reasons	5b882f2a4a, 20:40
4	shipping in every launcher	Windows done differently; Mac uncommitted
5, 6	hardening; host-side name lookup	not started

The times are author timestamps on 14 September. The four sprint commits were committed together at 20:45, and the plan they follow was written at 18:27 the same day. The timestamps say when each step was recorded, not how much work each took.

After the sprints

Commit	When	What it changed
50ef1a1bcd	14 Sep, 20:50	switched off, the whole doorbell window reads zero and ignores writes
3c523600ea	14 Sep, 20:57	the module refuses to load where there is no doorbell
20bf267db5	15 Sep, 00:41	the device builds on macOS
e8bdae8d8d	15 Sep, 04:46	the Windows release's switch: where the module file sits

What changed as it landed

A plan written before any code meets the code. The sprint commits record where the two differed, and each difference is a small lesson about RISC OS or QEMU:

Plan against landed

The plan said	What landed, and why
36 socket SWIs	35 , the number in the Internet module's own SWI table
unplug MbufManager with the rest	MbufManager stays : !Internet's start-up would try to load a 26-bit copy from the disc, fail, and stop before the name servers were set
find the doorbell at its address	map it with OS_Memory 13 : the physical address passes OS_ValidateAddress, and loading still takes a data abort
a plain request block	volatile : an optimised build folded the read of the result back to the value written a line earlier

Plan against landed, continued

The plan said	What landed, and why
would-block as an internal EWOULDBLOCK	an explicit HN_RC_RETRY answer: EWOULDBLOCK is kept for sockets the program set non-blocking
asynchronous notification in Sprint 3	pulled into Sprint 1: Resolver sets FIOASYNC on its socket and then waits only for the event
readability by peeking one byte	poll(), and WSAPoll() on Windows, from Sprint 2: a connect finishing shows as writability
events on "became readable"	events on "the reason changed": a socket with data that then breaks stays readable throughout

Plan against landed, continued again

The plan said	What landed, and why
a test module for Sprint 3	two BBC BASIC programs: BASIC is the only language on a machine with no toolchain
splice HostNet into every ROM	the Windows release loads it from the disc: no ROM rebuild on a machine without Python
a module that loads and stays dormant without a host	a module that declines to load: dormant, it would shadow a real Internet module
a device that answers its magic number when off	a window that reads zero when off: EtherGENET reads offset 0 as a revision number

The following chapters walk through each of these where it lives in the code.

How the code grew

Lines in each file at each commit:

Commit	Host <code>hostnet.c</code>	<code>hostnet.h</code>	Guest <code>hostnet.c</code>	<code>hostnet_entries.s</code>
23fc9630c5, Sprint 0	250	118	329	—
0f73da70af, Sprint 1	844	181	671	42
89013ef0f2, Sprint 2	1,340	192	698	42
5b882f2a4a, Sprint 3	1,578	214	730	42
d535505fad, as published	1,613	218	750	42

The guest module barely grew after Sprint 1, while the host device grew by about 770 lines. That is the design working as intended: each new socket verb is host code, and the guest's part, copying registers, waiting and raising events, was essentially complete once Sprint 1 had added the events.

3. A doorbell in the GENET window

Every HostNet request reaches the host through one small QEMU device. This chapter walks through where the device lives and why, its registers, the 64-byte request block, the commands and transport codes, the dispatcher that runs when the doorbell rings, and the switched-off state that makes the window look exactly as it did before HostNet existed. It ends by comparing the device with HostFS's doorbell.

A known hole at a known address

The Raspberry Pi 4's Ethernet controller, GENET, sits in the BCM2711's low peripheral window at guest physical address 0xFD580000. The emulator does not model it. Until HostNet, that 64 KB window held an *unimplemented device*: a QEMU stand-in that reads as zero. Its only purpose was that RISC OS's EtherGENET driver, reading the controller's revision register, would get zero and decline politely instead of taking an abort. A commit on 10 September, [5a61a7881c](#), had moved that stand-in to the address where RISC OS's hardware layer actually looks for GENET.

With the guest out of networking, EtherGENET is unplugged and the stand-in has nothing left to do. The header of `include/hw/misc/hostnet.h` describes what that leaves: "the 64 KB is a known-sized hole at a known address that RISC OS maps the same way it maps the HostFS doorbell." The same comment is careful to add that occupying it "is not modelling GENET; it is using the hole."

The wiring

This is the wiring, in `hw/arm/bcm2838_peripherals.c`:

```
/*
 * The GENET Ethernet MAC is not modelled, and with HostNet it never
 * will be: the guest does no networking below the socket layer, so
 * there is no link for a MAC to drive and EtherGENET is unplugged.
 * What stood here was an unimplemented device present only so that
 * EtherGENET's revision read returned zero rather than taking an
 * external abort. Its window is now HostNet's doorbell, same
 * address, same size. Low peripheral window (0xFD580000 to the
 * guest) because RISC OS maps that on request with OS_Memory 13,
 * the way the HostFS doorbell at 0xFD400000 is reached; a page in
 * the FE00 section aborts unless the HAL asked for that exact page.
 */
object_initialize_child(OBJECT(s), "hostnet", &s->hostnet, TYPE_HOSTNET);
if (!sysbus_realize(SYS_BUS_DEVICE(&s->hostnet), errp)) {
    return;
}
memory_region_add_subregion_overlap(&s->peri_low_mr, BCM2711_GENET_OFFSET,
    sysbus_mmio_get_region(SYS_BUS_DEVICE(&s->hostnet), 0), -1000);
```

The device has no interrupt line. GENET's two interrupt numbers remain defined and unused. Everything HostNet needs to tell the guest, it tells in the answer to a doorbell ring, including the wake-ups in chapter 6.

The registers

The window is 64 KB, and only the first four words mean anything. Every access must be a 4-byte, little-endian word:

Offset	Register	Read	Write
0x00	HN_MAGIC	'HNET', 0x54454E48	ignored
0x04	HN_VERSION	1	ignored
0x08	HN_FEATURES	bit 0, HN_FEATURE_SOCKETS	ignored
0x0C	HN_CMD	0	a request block's guest logical address: the doorbell
0x10 to the end	none	0	ignored

The value written to `HN_CMD` is a logical address, not a physical one. The module writes the address of its request block as its own code sees it, and the host walks the guest's page tables to find it. Chapter 4 describes that walk.

The request block

A request is a 64-byte header in guest memory, followed for one command by a list of results. The header deliberately mirrors HostFS's version 1 header, "so the two modules' transports read the same way":

Offset	Field	Direction	Meaning
+0	CMD	guest to host	0 PING, 1 SWI, 2 POLL
+4	SEQ	guest to host	a sequence number, kept by the host
+8	RC	host to guest	the transport result; the guest pre-fills 0xFFFFFFFF
+12	ERRNO	host to guest	a 4.BSD errno, 0 for none

The request block, continued

Offset	Field	Direction	Meaning
+16	SWI	guest to host	which socket SWI, 0 to 34
+20 to +51	R0 to R7	guest to host	the SWI's registers, verbatim
+52	RESULT	host to guest	R0 on success; the magic for PING; a count for POLL
+64 onwards	the poll list	host to guest	up to 16 words, for POLL only

Two details matter. The `RC` field starts as `0xFFFFFFFF`, which no answer uses, so a request the host never answered is visible as such. And socket errors do not use `RC` at all, as the next table explains.

Commands and transport codes

RC	Name	Meaning
0	HN_RC_OK	the call was made; <code>ERRNO</code> says whether it succeeded
1	HN_RC_BADCMD	an unknown command
2	HN_RC_BADSWI	a SWI number of 35 or more
3	HN_RC_BADADDR	a guest address would not translate
4	HN_RC_NOSOCKETS	the device is present but sockets are switched off
5	HN_RC_RETRY	the call would block on a socket the guest believes is blocking: wait and ring again

The comment on `HN_RC_OK` states the rule: a socket call that fails for an ordinary networking reason, such as a refused connection, "is not a transport failure, it is the answer". It comes back as `HN_RC_OK` with `ERRNO` set. The other codes mean the transport itself could not do what was asked, and `HN_RC_RETRY` is not a failure at all. Chapter 4 is about that last one.

The dispatcher

`hostnet_write` passes a write to `HN_CMD` to `hn_ring`, with the block's address. It runs on the guest CPU's thread, inside the memory write, holding QEMU's global lock. It starts by reading the header:

```
static void hn_ring(HostNetState *s, uint64_t base)
{
    uint32_t cmd = hn_ld32(base + HN_HDR_CMD);
    uint32_t regs[8];
    HNReply r = { .result = 0, .err = 0, .rc = HN_RC_OK };
    uint32_t swi;

    s->seq = hn_ld32(base + HN_HDR_SEQ);
    ...
    s->rung = true;
    ...
}
```

The path for a socket SWI

After `PING` and `POLL` have been answered, a socket SWI is checked, made, and answered:

```
swi = hn_ld32(base + HN_HDR_SWI);
if (swi >= HN_SWI_COUNT) {
    hn_st32(base + HN_HDR_RC, HN_RC_BADSWI);
    return;
}
if (!vmch_guest_rw(base + HN_HDR_REGS, regs, sizeof(regs), false)) {
    hn_st32(base + HN_HDR_RC, HN_RC_BADADDR);
    return;
}
if (!s->enabled) {
    hn_st32(base + HN_HDR_RC, HN_RC_NOSOCKETS);
    return;
}

errno = 0;
hn_do_swi(s, swi, regs, &r);
...
hn_st32(base + HN_HDR_RESULT, (uint32_t)r.result);
hn_st32(base + HN_HDR_ERRNO, r.err);
hn_st32(base + HN_HDR_RC, r.rc);
}
```

The elided parts answer `PING` and `POLL` and reject unknown commands before the SWI path, and write the trace line after it. `PING` writes the magic number into `RESULT`, which is how `*HostNetPing` checks that the host is there. `POLL` runs the wake-up scan from chapter 6.

Every verb fills in an `HNReply` of three fields, the result, the errno and the transport code, and the dispatcher writes all three back in one place. The `RC` word is written last. The guest reads it as soon as the memory write that rang the doorbell returns.

The rung flag

`s->rung` is set on every ring and cleared on reset:

Where	What it means
set in <code>hn_ring</code>	the module has rung since the machine last started
cleared in <code>hostnet_reset</code>	a restarted machine has not rung yet
read by the Windows window menu	whether the running RISC OS is using HostNet, whatever the module file's folder now says

The guest rings fifty times a second once the module is loaded, so the flag becomes true within one tick. Chapter 8 shows the menu using it.

Switched off: indistinguishable from before

The device's `sockets` property is off unless a launcher passes `-global hostnet.sockets=on`. The first version answered its magic number even when off, and only refused socket calls. `50ef1a1bcd` changed that, because the window's old zero reading had a job to do:

```
/*
 * Switched off, the whole window reads zero – not just the feature bits.
 *
 * This window used to hold an unimplemented device, and its reading as
 * zero was load-bearing rather than incidental: EtherGENET reads the GENET
 * revision register, which is at offset 0, and takes zero to mean an
 * unknown controller it should decline. Answering 'HNET' there would give
 * it a revision it has never heard of on a machine that has not asked for
 * HostNet at all.
 * ...
 */
static uint64_t hostnet_read(void *opaque, hwaddr offset, unsigned size)
{
    HostNetState *s = HOSTNET(opaque);

    if (!s->enabled) {
        return 0;
    }
    ...
}
```

Writes go nowhere

The write handler applies the same rule to the doorbell itself:

```
static void hostnet_write(void *opaque, hwaddr offset, uint64_t value,
                          unsigned size)
{
    HostNetState *s = HOSTNET(opaque);

    /* Switched off, writes go nowhere: a machine that has not asked for
     * HostNet must not be able to reach it by writing to the window. */
    if (s->enabled && offset == HN_CMD) {
        hn_ring(s, value);
    }
}
```

The window, read back

The commit records the three identification registers read from the guest in both states:

State	HN_MAGIC	HN_VERSION	HN_FEATURES
sockets off	0x00000000	0x00000000	0x00000000
sockets on	0x54454E48	0x00000001	0x00000001

The stock CMOS unplugs EtherGENET, so nothing was reading the register. The commit calls that "luck rather than a guarantee": a machine booted with a different CMOS would have found it. Off now also means unreachable. With sockets off, no guest program can make the host do anything by writing to this window.

Reset and snapshots

A machine reset abandons every socket the guest held, so `hostnet_reset` closes them all and clears the per-socket flags, the sequence number and `rung`. It leaves `enabled` alone, so a switch turned on in the session stays on.

The saved-state description migrates only the sequence number. Its comment explains the choice: "A host socket cannot be carried into a saved image, so a `-loadvm` restore finds every descriptor gone and answers EBADF. Pretending otherwise would hand the guest numbers that connect to nothing."

Two doorbells compared

HostNet copied HostFS's transport, but shares only one function with it. The table compares the two devices:

HostFS and HostNet

	HostFS, <code>vmchannel</code>	HostNet, <code>hostnet</code>
address and size	0xFD400000, 16 KB, plus a one-page alias at 0xFE005000	0xFD580000, 64 KB, no alias
magic	'VMCH'	'HNET'
header	64 bytes; R0 to R7 at +20	64 bytes; R0 to R7 at +20, RESULT at +52
addresses	guest logical, walked on the host	the same, through the same function

HostFS and HostNet, continued

	HostFS, vmchannel	HostNet, hostnet
shared code	owns vmch_guest_rw	calls it
state and sequence numbers	its own	its own
reset	flushes and closes open files, since a74bfcf41d	closes sockets, since Sprint 1

The Windows release adds one more dependency: HostNet's switch is a file on the HostFS share, so the menu that moves it needs a machine with a share. Chapter 8 covers that.

4. Never blocking the host

A socket call that waits, for data, a connection or a peer, is ordinary in any program. For HostNet it is the central difficulty, because the host answers from a place where it must not wait at all. This chapter explains that rule, then walks through how the host reaches the program's buffers, the retry answer that moves all waiting into the guest, the connect state machine, readiness on POSIX and on Windows, and `Select`, which is split between the two.

The rule

The comment at the top of `hw/misc/hostnet.c` states it:

```
/*
 * ...
 * Why the host must never block here: this runs synchronously inside the
 * vCPU's MMIO write, under the BQL. A blocking recvfrom() would freeze
 * the display, the monitor and the machine. So every host socket is
 * non-blocking, and an operation that would block on a socket the guest
 * believes is blocking comes back as HN_RC_RETRY – the guest spins its
 * own wait loop, which is exactly what it does today (tsleep() in the
 * Internet module's lib/c/unixenv spins on UpCall 6 and Portable_Idle:
 * there is no scheduler to sleep on).
 */
```

The doorbell handler runs on the thread of the guest CPU that wrote to the register, while that CPU is stopped in the middle of the write. It also holds QEMU's big lock, which the display, the monitor and every other device need. A host call that blocked for a second would stop the whole emulator for a second.

The last sentence of the comment is the key to the solution. RISC OS has no scheduler to put a waiting caller to sleep on. The Internet module's own blocking calls wait by looping: they offer the machine to anyone who can use it and check again. HostNet keeps that loop, and only moves the check to the host.

Reaching the program's buffers

A socket call's registers hold pointers into the calling program's memory: the data to send, the buffer to receive into, the address structure. The host reads and writes them with `vmch_guest_rw`, a function exported from the HostFS device. Its contract, in `include/hw/misc/vmchannel.h`:

```
/*
 * Read or write guest memory by its *logical* address, walking the
 * guest MMU a page at a time. Exported because HostNet (hw/misc/
 * hostnet.c) needs exactly this and for exactly the same reason: the
 * addresses in a request block are the ones the application passed,
 * so a socket buffer is read where it lies. It will not fault a page
 * in – an address not mapped right now fails rather than being
 * invented. Call only from a doorbell handler, where current_cpu is
 * the vCPU that rang and the BQL is held.
 */
bool vmch_guest_rw(uint64_t addr, void *buf, uint32_t len, bool is_write);
```

The walk translates one page at a time and copies through QEMU's physical memory map. The [HostFS walkthrough](#) covers how it works and why it will not fault a page in. For HostNet it means there is no copy in the guest: the module passes the program's own pointers, and the host reads the data where the program left it. On the host, a send copies the data into a temporary buffer of at most 1 MiB before calling `send`, and a receive copies out of one.

Always non-blocking underneath

Every host socket is switched to non-blocking mode the moment it is created or accepted. The two platforms do it differently:

```
static void hn_set_nonblock(int fd)
{
#ifdef _WIN32
    unsigned long one = 1;
    ioctlsocket(fd, FIONBIO, &one);
#else
    int fl = fcntl(fd, F_GETFL);
    if (fl >= 0) {
        fcntl(fd, F_SETFL, fl | O_NONBLOCK);
    }
#endif
}
```

On Windows `ioctlsocket` is one of the calls QEMU wraps, so it accepts the C runtime descriptor that QEMU's socket functions hand out. The comment above the function notes that failure is not checked, because a caller could do nothing useful with a blocking socket anyway.

What the *program* asked for is recorded separately. `Socket_Ioctl` with `FIONBIO` changes nothing on the host; it sets `nonblock[]` for that descriptor, and that flag decides what a would-block means.

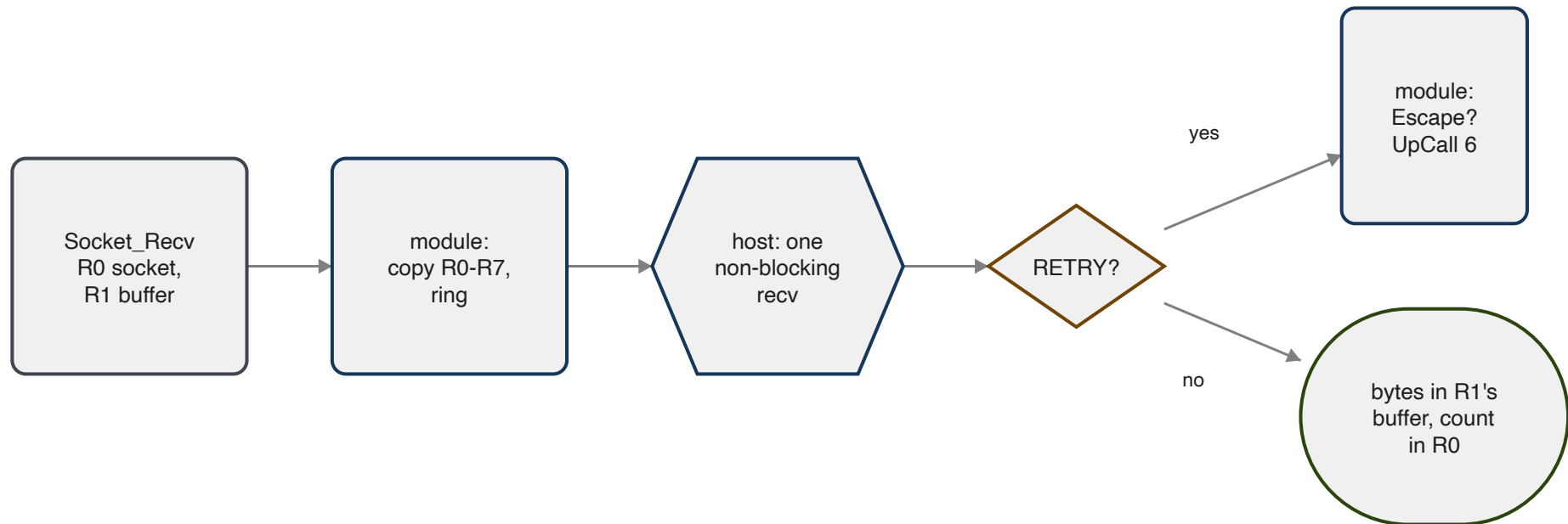
The retry answer

When a host call reports that it would block, the verb calls `hn_wouldblock`:

```
static void hn_wouldblock(HostNetState *s, HNReply *r, uint32_t id)
{
    if (id < HN_MAX_SOCKETS && !s->nonblock[id]) {
        r->rc = HN_RC_RETRY;
        return;
    }
    hn_errset(r, ROS_EWOULDBLOCK);
}
```

A program that set its socket non-blocking gets `EWOULDBLOCK`, exactly as it would from the Internet module, and deals with it itself. A program that left its socket blocking expects the call not to return until it can complete. For that program the host answers `HN_RC_RETRY`, which is not an answer at all: it tells the module to wait and ring again. The header's comment on the code says where that leaves RISC OS's blocking semantics: "That is where RISC OS's blocking semantics actually live."

One blocking Socket_Recv, round trip



The loop from "wait" back to "ring" is the module's `hn_wait_ring`, shown in chapter 7. Each turn checks for Escape, which ends the wait with `EINTR`, and makes `OS_UpCall 6`, which lets a `TaskWindow` run other tasks while this one waits. Outside a `TaskWindow` nothing takes up the offer, so the loop spins. As the module's comment says, that "is what a blocking RISC OS SWI has always been".

The Sprint 3 test shows the loop working. `listen.bas` makes a blocking `Socket_Accept_1` and waits for the host to connect. The trace shows seven `HN_RC_RETRY` answers, then the accepted connection.

Connect, without connecting twice

A non-blocking `connect` normally returns at once with "in progress", and the connection completes later. The obvious retry, calling `connect` again, only reports "already in progress" and says nothing about whether the connection worked. So `hn_connect` remembers that a connection is in flight and, on the next call, asks instead:

```
if (s->connecting[R[0]]) {
    int err = 0;
    socklen_t el = sizeof(err);

    if (!(hn_ready(fd, HN_W) & (HN_W | HN_X))) {
        hn_wouldblock(s, r, R[0]);    /* still on its way */
        return;
    }
    s->connecting[R[0]] = false;
    if (getsockopt(fd, SOL_SOCKET, SO_ERROR, (char *)&err, &el) < 0) {
        hn_fail(r);
        return;
    }
    if (err) {
        errno = err;
        hn_fail(r);
        return;
    }
    hn_ok(r, 0);
    return;
}
```

The connect state machine

Call	Host state	What the guest gets
first	connect says in progress; connecting set	EINPROGRESS if non-blocking, HN_RC_RETRY if blocking
later, not yet writable	still connecting	EWOULDBLOCK if non-blocking, HN_RC_RETRY if blocking
later, writable or failed	connecting cleared; SO_ERROR read	success, or the connection's error translated to a RISC OS errno

A socket becomes writable when its connection completes, and reports an error condition when it fails, which is why the test is for either.

Readiness, on two platforms

Both `connect` and `Select` need to ask whether a socket is ready without waiting. Sprint 1 answered "is it readable?" by peeking one byte with `recv`. That needed no special handling on Windows, but it cannot answer "is it writable?", and Sprint 2 needed that for `connect`. The answer since then is `poll`, or `WSAPoll` on Windows:

```
static short hn_revents(int fd, short ev)
{
#ifdef _WIN32
    WSAPOLLFD p;

    p.fd = (SOCKET)_get_osfhandle(fd);
    p.events = ev;
    p.revents = 0;
    if (p.fd == (SOCKET)INVALID_HANDLE_VALUE || WSAPoll(&p, 1, 0) <= 0) {
        return 0;
    }
    return p.revents;
#else
    struct pollfd p;

    p.fd = fd;
    p.events = ev;
    p.revents = 0;
    if (poll(&p, 1, 0) <= 0) {
        return 0;
    }
    return p.revents;
#endif
}
```

On Windows, QEMU's socket functions return C runtime file descriptors, each made from a Windows `SOCKET` with `_open_osfhandle`. `WSAPoll` wants the `SOCKET` itself, so `_get_osfhandle` recovers it. The timeout is zero on both platforms: the question is always "now?", never "when?".

`hn_ready` turns the raw answer into what `Select` may report. Its comment explains one deliberate choice: an error or hang-up counts as *readable* as well as exceptional, so that a program waiting to read from a connection that has died wakes up and gets the error from the read, rather than waiting for ever.

Select, split in two

`Socket_Select` takes three descriptor sets and a timeout. The host cannot wait out the timeout, so the call is split. The host's half makes one pass over the sets:

```
/*
 * Only written back when something is ready. A zero answer leaves the
 * caller's input sets untouched, which is what lets the module ask
 * again without rebuilding them – and the module has to ask again,
 * because the timeout is the caller's and the host cannot wait.
 */
if (count) {
    for (k = 0; k < 3; k++) {
        if (R[1 + k]) {
            vmch_guest_rw(R[1 + k], out[k], HN_FDSET_BYTES, true);
        }
    }
}
hn_ok(r, (int32_t)count);
```

Before that point, the function reads each 32-byte set from guest memory, answers `EBADF` for a bit set on a closed descriptor, and counts one for each ready bit, as BSD does.

The module's half, `hn_select`, owns the timeout. It computes a deadline from the program's `timeval` using `OS_ReadMonotonicTime`, then loops: ring the host once; return if the count is non-zero; return 0 if the deadline has passed; check Escape; offer the machine with `OS_UpCall 6`; ring again. The deadline test is a signed comparison, so a wrap of the centisecond counter cannot turn a short wait into a very long one.

Who does what in Select

Part	Host, hn_select in hostnet.c	Guest module, hn_select
the descriptor sets	read once per ring; written back only when something is ready	passed through untouched
the timeout	never seen	a deadline in centiseconds, checked each turn
waiting	never	the loop, with Escape and OS_UpCall 6
the result	the count of ready bits	R0: the count, or 0 at the deadline

5. The verbs

The host device implements the socket calls one by one, and the sprints added them in groups: UDP first, then TCP, then listeners and the message calls. This chapter lists all 35 SWIs with the sprint that served each one. It then walks through the translation layer that sits between a RISC OS program and the host's socket API: socket addresses, message flags, socket options, ioctl's and error numbers. It ends with the limits the device imposes and the differences between Windows and POSIX hosts.

The dispatch

`hn_do_swi` is a single `switch` on the SWI number, grouped in the source by sprint. The Sprint 1 group is where every other verb started:

```
static void hn_do_swi(HostNetState *s, uint32_t swi, uint32_t *R, HNReply *r)
{
    switch (swi) {
        case HN_SWI_CREAT:      hn_creat(s, R, r); break;
        case HN_SWI_BIND:      hn_bind(s, R, r); break;
        case HN_SWI_SENDTO:    hn_sendto(s, R, r); break;
        ...
        /* Sprint 2: TCP. */
        case HN_SWI_CONNECT:    hn_connect(s, R, r); break;
        ...
        default:
            hn_errset(r, ROS_EOPNOTSUPP);
            break;
    }
}
```

Anything without a case answers `EOPNOTSUPP`, which the module turns into a RISC OS error. That was the whole of Sprint 0: every call reached the host and came back "not supported". Its commit points out that `ARP -s` reporting "ARP: socket" was "the whole path working".

All 35 SWIs

The SWI numbers are fixed by the Internet module's interface: &41200 plus the SWI's position in this order, which is also the order of `swis.txt` :

```
Creat, Bind, Listen, Accept, Connect, Recv, Recvfrom, Recvmsg, Send, Sendto, Sendmsg, Shutdown, Setsockopt, Getsockopt, Getpeername,
Getsockname, Close, Select, Ioctl, Read, Write, Stat, Readv, Writev, Gettsize, Sendto, Sysctl, Accept_1, Recvfrom_1, Recvmsg_1, Sendmsg_1,
Getpeername_1, Getsockname_1, InternalLookup, Version
```

So `Socket_Creat` is &41200, `Socket_Close` is &41210 and `Socket_Version` is &41222. The six calls without `_1` that also have a `_1` form are the Internet 4 versions.

Served since

Sprint	SWIs
1	Creat, Bind, Sendto, Recvfrom, Recvfrom_1, Close, Ioctl, Select for reading, Gettsize, Version, Sysctl
2	Connect, Send, Recv, Read, Write, Readv, Writev, Shutdown, Setsockopt, Getsockopt, the four name calls, Select in full
3	Listen, Accept, Accept_1, Sendmsg, Sendmsg_1, Recvmsg, Recvmsg_1
never	Stat, Sendto, InternalLookup: EOPNOTSUPP

Descriptors are slots

A RISC OS socket descriptor is a small number, and programs build `fd_set` bitmaps indexed by it. HostNet keeps a table of 256 slots, the same size as the Internet module's socket table, which is also what `Socket_Gettsize` reports. Each slot holds a host socket or `-1`, and the guest's descriptor *is* the slot number. New sockets take the lowest free slot, as the Internet module does, because, as the comment on `hn_alloc` puts it, "programs do notice if descriptors come back in a different order".

What each slot records

Field	Set by	Used for
<code>fds[]</code>	<code>Creat, Accept</code>	the host socket, or <code>-1</code>
<code>nonblock[]</code>	<code>Ioctl FIONBIO</code>	whether a would-block is <code>EWOULDBLOCK</code> or <code>HN_RC_RETRY</code>
<code>async[]</code>	<code>Ioctl FIOASYNC</code>	whether wake-ups raise Internet Event 19
<code>owner[]</code>	<code>Ioctl FIOSETOWN</code>	recorded and returned; not otherwise used
<code>woke[]</code>	the wake-up poll	the last event reason reported (chapter 6)
<code>connecting[]</code>	<code>Connect</code>	a non-blocking connection is in flight (chapter 4)

Socket addresses, three shapes

A RISC OS socket address comes in two layouts, and the host's is a third. The new layout, used by the `_1` SWIs, is a length byte, a family byte, the port, the address and eight zero bytes. The old layout has a 16-bit family word in place of the first two bytes. macOS's own layout has a length byte and Windows' does not. Ports and addresses are in network byte order in all of them, so they are copied as bytes. `hn_sa_in` builds a host address from either guest layout:

```
static bool hn_sa_in(uint64_t addr, uint32_t len, struct sockaddr_in *sin)
{
    uint8_t g[HN_SA_LEN];

    if (len < 8 || len > HN_SA_LEN) {
        return false;
    }
    memset(g, 0, sizeof(g));
    if (!vmch_guest_rw(addr, g, len, false)) {
        return false;
    }
    memset(sin, 0, sizeof(*sin));
    sin->sin_family = AF_INET;
    memcpy(&sin->sin_port, g + 2, 2);
    memcpy(&sin->sin_addr, g + 4, 4);
    return true;
}
```

Both guest layouts put the port at offset 2 and the address at offset 4, so reading them needs no knowledge of which layout it is. Writing an address back does need to know: `hn_sa_out` sets a length byte and a family byte for the new layout, and a 16-bit family word for the old one.

A worked address

`listen.bas` binds port 9000 on every interface, in the new layout. The bytes it writes, and what the host builds from them:

Offset	Bytes in guest memory	Meaning	Host <code>sockaddr_in</code>
+0	16	length (new layout)	not copied
+1	2	family, <code>AF_INET</code>	set to <code>AF_INET</code> whatever it says
+2	35, 40	port 9000 in network order: $35 \times 256 + 40$	<code>sin_port</code> , copied as two bytes
+4	0, 0, 0, 0	address 0.0.0.0, any interface	<code>sin_addr</code> , copied as four bytes

Message flags: translated, never passed through

The flag word of `send`, `recv` and the message calls is translated bit by bit, and unknown bits are dropped. NetSurf's DNS library, c-ares, sets `0x400`, its own `MSG_NOSIGNAL`, on every send. The host rejected the unknown bit with `EOPNOTSUPP`, and, in Sprint 2's words, "every HTTPS fetch failed at the first byte while the trace showed a perfectly good connected socket":

```
static int hn_msgflags(uint32_t g)
{
    int h = 0;

    if (g & 0x1) {
        h |= MSG_OOB;
    }
    if (g & 0x2) {
        h |= MSG_PEEK;
    }
    if (g & 0x4) {
        h |= MSG_DONTROUTE;
    }
#ifdef MSG_WAITALL
    if (g & 0x40) {
        h |= MSG_WAITALL;
    }
#endif
#ifdef MSG_NOSIGNAL
    h |= MSG_NOSIGNAL;
#endif
    return h;
}
```

`MSG_NOSIGNAL` is always added where the host has it, because a `SIGPIPE` from a guest writing to a closed connection would kill the emulator, not the guest. For macOS, which the source notes has no `MSG_NOSIGNAL`, `hn_no_sigpipe` sets `SO_NOSIGPIPE` on every socket instead.

Socket options, by name

The numbers agree on RISC OS, Windows and macOS, but options are still translated by name. An unknown one is accepted and ignored: "a program that cannot set SO_DEBUG should carry on, not fail".

Level	Options served, by guest number	On the host
SOL_SOCKET, 0xFFFF	REUSEADDR 0x4, KEEPALIVE 0x8, BROADCAST 0x20, LINGER 0x80, OOBINLINE 0x100, SNDBUF 0x1001, RCVBUF 0x1002, ERROR 0x1007, TYPE 0x1008	the same names; SO_ERROR translated
IPPROTO_TCP, 6	TCP_NODELAY 0x1	TCP_NODELAY
anything else	any	set: ignored; get: ENOPROTOPT

Option values of up to 32 bytes are passed through otherwise unchanged.

Ioctl's

Five requests are served, by the guest's own numbers:

Request	Number	What HostNet does
FIONBIO	0x8004667E	records the program's non-blocking choice
FIONREAD	0x4004667F	writes 1 if the socket is readable, else 0
FIOASYNC	0x8004667D	records whether to raise Internet Event 19 for this socket
FIOSETOWN, FIOGETOWN	0x8004667C, 0x4004667B	records and returns the owner
anything else		EOPNOTSUPP

`FIOASYNC` has a story. In Sprint 1 it was going to be refused until the events existed, but the stock Resolver module sets it immediately after creating its socket, and closes the socket if the request fails. The comment in `hn_ioctl1` records the result of refusing it: it "is why DNS did nothing at all the first time Sprint 1 was tested".

Sysctl, for one line of the boot

`Socket_Sysctl1` reads and sets the network stack's tunable values. HostNet has none, but it cannot refuse the call. `!Internet`'s start-up sets one value, the UDP checksum switch, under `CheckError`, so a refusal would stop the boot before the file that sets the name servers. HostNet accepts every `Sysctl1`, and a request to read a value gets a length of zero back.

Error numbers, by name

The host reports failures through `errno`, whose numbers are the host's: native on macOS, and on Windows the C runtime's, which QEMU's wrappers set from Winsock's error. RISC OS programs expect 4.4BSD numbers. `hn_errno` therefore maps by *name*, with a `case` for each error a socket call can produce, and returns RISC OS's numbers:

The RISC OS numbers

Group	Errors and their RISC OS numbers
general	EINTR 4, EBADF 9, EACCES 13, EFAULT 14, EINVAL 22, EMFILE 24
would block	EWOULDBLOCK 35, also from EAGAIN; EINPROGRESS 36; EALREADY 37
the socket and protocol	ENOTSOCK 38, EDESTADDRREQ 39, EMSGSIZE 40, EPROTOTYPE 41, ENOPROTOOPT 42, EPROTONOSUPPORT 43, EOPNOTSUPP 45, EAFNOSUPPORT 47
addresses and networks	EADDRINUSE 48, EADDRNOTAVAIL 49, ENETDOWN 50, ENETUNREACH 51, EHOSTUNREACH 65
connections	ECONNABORTED 53, ECONNRESET 54, ENOBUFS 55, EISCONN 56, ENOTCONN 57, ETIMEDOUT 60, ECONNREFUSED 61

Any host error not in the list becomes `EINVAL`. The module then builds the RISC OS error block from the number: `&20E00` plus the `errno`, with the error's name, such as `CONNREFUSED`, as the message.

Limits

Limit	Value	Why
sockets per machine	256	the Internet module's own table size
bytes per call	1 MiB	"a bound on what the host will allocate"
readv/writev entries; option value	1024; 32 bytes	bounds on a bad count or length
fd_set; events per poll	32 bytes; 16	one bit per slot; the reply list
address family	IPv4 only	AF_INET6 answers EAFNOSUPPORT

Windows and POSIX

The first working build of the device was on Windows. 20bf267db5 made it build on macOS, and the two hosts differ in a handful of places:

The two hosts

Concern	Windows	macOS and other POSIX hosts
descriptor	a C runtime descriptor wrapping a SOCKET	a native descriptor
non-blocking	ioctlsocket, wrapped by QEMU	fcntl with O_NONBLOCK
readiness	WSAPoll on _get_osfhandle(fd)	poll, with <poll.h> included explicitly for macOS
closing	Winsock's closesocket	close, through #define closesocket close
SIGPIPE	not applicable	MSG_NOSIGNAL where it exists, SO_NOSIGPIPE on macOS

6. Internet Event 19 without an interrupt

Some RISC OS programs never ask whether a socket has data. They ask to be told, by Internet Event 19, and then wait. The Internet module raised that event from its own receive path. HostNet has no receive path in the guest, and the host device has no interrupt and no way to call into the guest. This chapter shows how the module asks instead, fifty times a second: a ticker, a callback, a doorbell command called `POLL`, and an event for each socket that woke. It ends with the Sprint 3 change from "became readable" to "the reason changed", and the test that proves it.

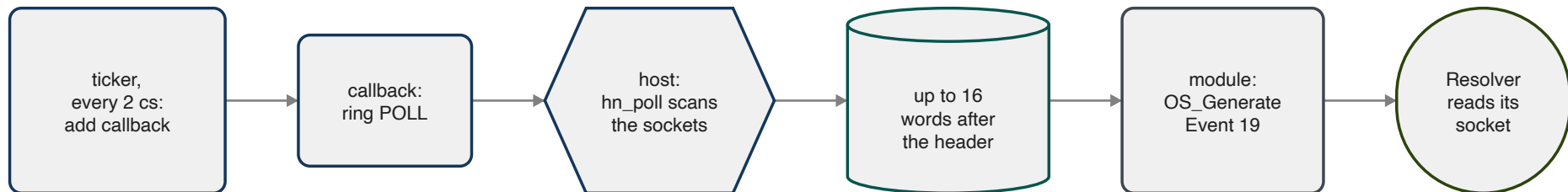
Why events are not optional

The comment above the module's event code gives the reason, and it is a real program, not a hypothetical one:

```
/*
 * Internet Event 19 is how a RISC OS program finds out that a socket has
 * woken. The Internet module raised it from its own receive path; there
 * is no receive path here, and the host cannot call into the guest, so the
 * module asks on a ticker and raises the events itself.
 *
 * This is not a nicety. The stock Resolver sets FIOASYNC, sends its
 * query, and then does nothing at all until an event arrives – it never
 * polls and never reads. Without this, DNS queries go out, answers come
 * back to the host, and the guest never asks for them.
 */
```

That is why events, planned for Sprint 3, were built in Sprint 1. Sprint 1's test was a name lookup through the stock Resolver, and it could not pass without them.

The chain, once per tick



Each step runs in a different context, and that is the reason for the chain's shape. A ticker routine runs with interrupts disabled and very little of the operating system available, so it does the one thing that is safe there: it adds a callback. A callback runs later, when the operating system is in a normal state, so it can ring the doorbell and raise events. The veneer comments in `hostnet_entries.s` note that this is the same shape the Internet module uses for its own deferred work.

The ticker and the callback

`hostnet_init` registers the ticker with `OS_CallEvery`, every 2 centiseconds. The C code it reaches does almost nothing:

```
void hostnet_c_tick(void)
{
    tick_count++;
    if (!live || cb_pending) {
        return;
    }
    cb_pending = 1;
    os_add_callback((uint32_t)hostnet_callback, static_base());
}
```

The callback

The callback asks the host, and raises one event for each socket in the answer:

```
void hostnet_c_callback(void)
{
    uint32_t i, n;

    cb_pending = 0;
    cb_count++;
    if (!live) {
        return;
    }
    ...
    pollreq[H_CMD] = HN_CMD_POLL;
    pollreq[H_SEQ] = ++seq;
    pollreq[H_RC] = 0xFFFFFFFFu;
    hn_reg(HN_CMD / 4) = (uint32_t)pollreq;
    if (pollreq[H_RC] != HN_RC_OK) {
        return;
    }
    n = pollreq[H_RESULT];
    if (n > HN_POLL_MAX) {
        n = HN_POLL_MAX;
    }
    for (i = 0; i < n; i++) {
        uint32_t w = pollreq[H_WORDS + i];
        ...
        os_generate_event(19, (w >> 8) & 0xFFu, w & 0xFFu, w >> 16);
        event_count++;
    }
}
```

Clearing `cb_pending` first lets the next tick queue another callback, and the elided loop zeroes the header. The callback uses its own request block, `pollreq`, not the one the SWIs use. The comment explains why: "a callback can land between a SWI writing the doorbell and reading its answer, and

sharing the block would let one overwrite the other."

Two veneers, and a wrong SWI number

The ticker and the callback are entered with R12 holding the value passed when they were registered. The module is built position-independent for its data, so its C code finds its variables through R9. The module passes its static base as the R12 value, and a two-instruction veneer in `hostnet_entries.s` moves it into R9:

```
hostnet_tick:
    stmfd    sp!, {r0-r3, r9, lr}
    mov     r9, r12                @ static base, as passed to OS_CallEvery
    bl      hostnet_c_tick
    ldmfd    sp!, {r0-r3, r9, lr}
    mov     pc, lr
```

The first version of Sprint 1 registered no ticker at all, and said nothing about it. `OS_CallEvery` is SWI &3C; the code called &3D, which is `OS_RemoveTickerEvent`, and removing a ticker that was never added succeeds silently. The commit credits the new trace and `*HostNetInfo`'s counters with finding it. From the host's side, "the guest never asked" and "the guest asked and there was nothing" look the same, and a tick counter is what tells them apart. The Sprint 2 commit adds that every SWI number in the module was then checked against the kernel's own table.

The host's answer

`hn_poll` scans every slot. It looks only at sockets whose program asked for `FIOASYNC`, and for each one it decides a *reason*:

```
if (s->fds[i] < 0 || !s->async[i]) {
    s->woke[i] = 0;
    continue;
}
re = hn_revents(s->fds[i], POLLRDNORM);
now = (re & (POLLRDNORM | POLLPRI | POLLERR | POLLHUP)) != 0;
...
if (re & (POLLERR | POLLHUP)) {
    reason = HN_EV_BROKEN;
} else if (re & POLLPRI) {
    reason = HN_EV_URGENT;
} else {
    reason = HN_EV_ASYNC;
}

if (!now) {
    reason = 0;
}
if (reason && reason != s->woke[i] && n < HN_POLL_MAX) {
    ...
    list[n++] = (port << 16) | (reason << 8) | (uint32_t)i;
}
s->woke[i] = (uint8_t)reason;
```

The elided lines explain the precedence and look up the socket's local port with `getsockname`. After the loop, the list is written into guest memory straight after the 64-byte header, and the count goes in `RESULT`.

The three reasons

The reasons are Internet Event 19's own, which the header describes as SIGIO, SIGURG and SIGPIPE "as a RISC OS program sees them":

Value	Name	Meaning	Precedence
1	HN_EV_ASYNC	something to read	lowest
2	HN_EV_URGENT	out-of-band data	middle
3	HN_EV_BROKEN	the connection has gone	highest: broken is broken whatever else is true

One word per event

Each word packs the event's three register values, "ready to use". A worked example, from the Sprint 3 trace below:

Field	Bits	Value	Becomes
local port	31–16	64641, 0xFC81	R3 of the event
reason	15–8	3, broken	R1 of the event
descriptor	7–0	1	R2 of the event
the word		0xFC810301	OS_GenerateEvent 19, 3, 1, 64641

The descriptor fits in eight bits because there are 256 slots. That leaves the middle byte free for the reason, as the header's comment points out.

Edges, not levels, and which edge

A readable socket stays readable until someone reads it. Reporting the *level* would raise the same event fifty times a second until the program got round to reading, so the poll reports *edges*: an event is raised only when something changes.

Sprint 1's edge was readability: not readable, then readable. Sprint 3 found the flaw in that, and the header's comment on `woke[]` describes it: "a socket that has data and then breaks stays readable throughout, so tracking readability alone reports the first wake and silently swallows the disconnection -- which is the one a program most needs to hear about."

The same socket, under both rules

A connection receives a line of data, then the peer closes it:

Moment	Readable?	Reason	Sprint 1 rule: readability changed	Sprint 3 rule: reason changed
connected, idle	no	0	no event	no event
a line arrives	yes	async	event , async	event , async
the peer closes, data unread	yes	broken	no event: still readable	event , broken
the program reads and closes	—	—	no event	no event

Under the Sprint 1 rule, a program that waits for events never learns the connection has gone. Under the Sprint 3 rule it gets both events.

The test: `evtest.bas`

`evtest.bas` is a guest client that does nothing on its own initiative. It connects to a listener on the host, asks for `FIOASYNC`, and then sits in an empty loop for nine seconds without touching the socket:

```
SYS "Socket_Creat", 2, 1, 0 TO s%
opt%!0 = 1
SYS "Socket_Ioctl", s%, &8004667D, opt% : REM FIOASYNC
SYS "Socket_Connect", s%, sa%, 16
PROClog("connected, async requested")
REM Let the events happen. A plain wait: nothing here touches the socket.
t% = TIME
REPEAT UNTIL TIME > t% + 900
SYS "Socket_Recv", s%, buf%, 256, 0 TO n%
```

A host program accepts the connection, sends a line, waits and closes. With `HOSTNET_TRACE` set, the host's trace shows the two edges, at poll 577 and poll 743:

```
hostnet: POLL #577 -> 1 ready fd1=async:64641
hostnet: POLL #743 -> 1 ready fd1=broken:64641
```

The Sprint 3 commit adds that the program then read its eight bytes "on the strength of the event alone".

The trace heuristic

`hn_poll` does not log every poll; fifty lines a second would bury everything else. It logs every poll that found something, the first three, and every 200th:

```
{ /* Every poll, not only the useful ones: "the ticker never
 * ran" and "the ticker ran and found nothing" look identical
 * from the guest, and they have completely different causes. */
static uint32_t polls;
if (n || polls < 3 || (polls % 200) == 0) {
    hn_trace("hostnet: POLL #%u -> %u ready%s\n", polls, n,
            n ? hn_reasons(list, n) : "");
}
polls++;
}
```

At 50 polls a second, every 200th is one line every four seconds: enough to show that the ticker is alive, and too few to get in the way.

What `*HostNetInfo` shows

The module's own counters answer the same question from the guest's side:

Counter	Counts	A problem it reveals
ticks	ticker entries	zero: the ticker was never registered, as in Sprint 1's first version
callbacks	callbacks run	far fewer than ticks: callbacks are not being delivered
events	Event 19s raised	zero while a program waits: the host has seen nothing wake

`*HostNetInfo` prints the doorbell's state and these three numbers on one line. `*HostNetPing` rings `PING` and prints nothing if the host answers with its magic number.

7. Internet 6.00 in 750 lines

The guest half of HostNet is one C file, one short assembly file and a list of SWI names. This chapter walks through it from build to shutdown: how it is built and why not with the usual RISC OS tools, how it finds the doorbell, the one keyword without which every request appeared to fail, the transport and the wait loop, how errors are returned, why it refuses to load on a machine without the doorbell, and the two ways it gets into a running machine.

What the module says it is

The header comment of `riscos-pi4/hostnet/hostnet.c` sets out the whole job:

```
/*
 * HostNet – the guest half.  Internet 6.00.
 *
 * It claims the Internet module's name and its SWI chunk (&41200), and
 * every one of the thirty-five socket SWIs does the same thing: copy R0-R7
 * into a request block, ring the doorbell at the GENET window, copy R0
 * back. It does not interpret the registers, does not know what a socket
 * is, and holds no state beyond the block itself. All the networking is
 * on the host (hw/misc/hostnet.c).
 *
 * ...
 */
```

Identity

Item	Value
title	Internet
help string	HostNet 6.00 (14 Sep 2026)
SWI chunk and prefix	&41200, Socket
SWIs	35, generated from swis.txt
star commands	*HostNetPing, *HostNetInfo
entry points	hostnet_init, hostnet_final; no service call handler

Where the ROM's Internet module is also present, the kernel gives the *later* claimant of a SWI chunk priority, so HostNet would win anyway. The comment is clear that this is "not the plan, it is the safety net". The plan is that the ROM's stack is not running: unplugged in CMOS when HostNet is spliced into the ROM, or replaced as HostNet loads from the disc.

Building it

`build-hostnet.sh` uses the same toolchain as the HostFS module beside it: ROSCC's module header generator and linker, clang targeting ARM, and ROSCC's small runtime. It is not built with the RISC OS Desktop Development Environment. The script's comment gives the reason: the module has to be able to run from ROM, and the usual module header generator and C library stubs both write into the module image during initialisation, which in ROM is a data abort.

The build, step by step

Step	Tool	Output
1. the SWI list	a short Python script over <code>swis.txt</code>	the C entry points and the header's SWI list
2. the module header	ROSCC's <code>gen_module.py</code>	<code>module_head.s</code>
3. the code	clang: the header, <code>hostnet_entries.s</code> , <code>hostnet.c</code>	object files
4. the runtime	clang: ROSCC's runtime sources	object files
5. the link	<code>rosc link --module</code>	<code>build/HostNet,ffa</code>

Generating the entry points

Step 1 is what keeps the SWI order right. The same list produces both the module header's table of names and the C entry points, so the two cannot disagree:

```
names = [n for n in open('swis.txt').read().split() if n]
nl = chr(10)
open('hostnet_swis.inc', 'w').write(
    '/* Generated by build-hostnet.sh from swis.txt -- do not edit.' + nl +
    ' * One entry point per socket SWI, in chunk order from &41200.' + nl +
    ' * Each is a veneer target: it does nothing but name its own number. */'
    + nl +
    nl.join(f'HN_ENTRY({i:02d}, {i}) /* Socket_{n} */'
           for i, n in enumerate(names)) + nl)
```

Each generated line expands, through the `HN_ENTRY` macro at the end of `hostnet.c`, into a function that calls `hn_call` with its own number.

The compiler flags

Flag	Why
<code>--target=armv8a-none-eabi -mcpu=cortex-a72</code>	the emulated processor, in 32-bit mode
<code>-ffreestanding -nostdlib -fno-builtin</code>	a module has no C library but the runtime linked in
<code>-fropi -frwpi</code>	position-independent code and data, found through R9
<code>-O2 -mno-movt</code>	optimised, and no <code>movw/movt</code> instruction pairs for loading constants
<code>-DROSTRT_STATIC_HEAP -DHEAP_BYTES=1024 -DARENA_BYTES=256</code>	a tiny static heap: "HostNet allocates nothing"

The heap comment finishes the thought: "every request goes in one static block, and every buffer it touches belongs to the caller." A local build of the module, made on 14 September and not committed, is 12,032 bytes.

Finding the doorbell

The first version of the module used the doorbell's physical address as if it were a logical one, and took a data abort during boot "with nothing on screen to say why". RISC OS maps peripherals into logical memory on request, so the address must come from `OS_Memory 13`, which maps a physical range permanently and returns where it now lives:

```
static volatile uint32_t *os_map_io(uint32_t phys, uint32_t size)
{
    register uint32_t r0 __asm("r0") = 13;
    register uint32_t r1 __asm("r1") = phys;
    register uint32_t r2 __asm("r2") = size;
    register uint32_t r3 __asm("r3");
    register uint32_t v __asm("r4");

    __asm volatile("swi 0x20068\n\tmrs r4, cpsr"
                  : "+r"(r0), "+r"(r1), "+r"(r2), "=r"(r3), "=r"(v)
                  :
                  : "r12", "lr", "cc", "memory");
    if (v & (1u << 28)) {
        return 0;
    }
    return (volatile uint32_t *)r3;
}
```

This is how every SWI in the module is made, with the same register-binding assembly form that ROSCC's runtime uses. The string holds the instructions; the lists after it bind the C register variables and name what the call may overwrite. SWI &20068 is the error-returning form of `OS_Memory`. The `mrs` after it copies the flags into R4, and bit 28 is V, set on an error.

The Sprint 0 commit adds a second trap: `OS_ValidateAddress` "happily calls the fixed address valid and the load still takes a data abort". A validity check is no substitute for asking the operating system to map the page.

The block that had to be volatile

The request block is a static array of 16 words, and its declaration carries a comment that is worth reading in full:

```
/*
 * volatile, and not as a formality: the host writes into this block
 * through the guest's MMU while the compiler believes nothing can have
 * touched it. Without it, an optimising build folds the read of the
 * result back to the poison value written a line earlier and every
 * request appears to fail. (HostFS avoids this only by building at -O0.)
 */
static volatile uint32_t req[H_WORDS] __attribute__((aligned(16)));
```

What the optimiser saw

Source line	What the compiler knows without volatile	What really happens
req[H_RC] = 0xFFFFFFFFu;	req[H_RC] now holds 0xFFFFFFFF	the same
hn_reg(HN_CMD / 4) = (uint32_t)req;	a write that stores req's address and changes nothing in it	the host runs the request and rewrites req[H_RC]
return req[H_RC];	still 0xFFFFFFFF, so return the constant	the host's answer, if the compiler reads memory

Without `volatile`, the compiler is entitled to skip the read and return 0xFFFFFFFF. Every request then looks unanswered. The keyword tells it that the memory can change behind its back, which is exactly the case.

The transport

Two small functions ring the doorbell. `hn_go` stamps the sequence number and the poison value, and writes the block's address:

```
static uint32_t hn_go(void)
{
    req[H_SEQ] = ++seq;
    req[H_RC] = 0xFFFFFFFFFu;      /* poison: an unanswered request shows */
    hn_reg(HN_CMD / 4) = (uint32_t)req;
    return req[H_RC];
}
```

One SWI into the block

`hn_ring_once` fills the header for one SWI and calls it:

```
static uint32_t hn_ring_once(uint32_t swi, unsigned *regs)
{
    uint32_t i;

    req[H_CMD] = HN_CMD_SWI;
    req[H_SWI] = swi;
    req[H_ERRNO] = 0;
    req[H_RESULT] = 0;
    for (i = 0; i < 8; i++) {
        req[H_REGS + i] = regs[i];
    }
    return hn_go();
}
```

Only R0 to R7 cross, which covers every argument any socket call takes. On the way back only R0 is written, because R0 is the only register the Internet module ever returns a value in.

The wait

`hn_wait_ring` is where a blocking socket call waits. Chapter 4 explained why the host cannot:

```
static _kernel_oserror *hn_wait_ring(uint32_t swi, unsigned *regs,
                                     uint32_t *rc_out)
{
    for (;;) {
        uint32_t rc = hn_ring_once(swi, regs);

        if (rc != HN_RC_RETRY) {
            *rc_out = rc;
            return 0;
        }
        if (os_escape()) {
            return sock_error(4);      /* EINTR */
        }
        os_yield();
    }
}
```

`os_yield` makes `OS_UpCall 6` with a pollword that never changes, which is what the Internet module's own sleep does. A TaskWindow claims the upcall, lets the desktop run, and returns; with nothing to claim it, the call returns at once. The comment says what the loop must never do: "spin *without* offering, which would stop a TaskWindow multitasking at all." `OS_UpCall 6` was added in Sprint 2.

From the answer to a result

`hn_call`, which every SWI entry point reaches, puts it together. It returns an error if the doorbell was never found, sends `Select` to its own loop (chapter 4), runs the wait, and turns the transport code and `errno` into a RISC OS result:

```
switch (rc) {
case HN_RC_OK:
    break;
case HN_RC_NOSOCKETS:
    return &err_nosock;
case HN_RC_BADSWI:
    return &err_badswi;
case HN_RC_BADADDR:
    return &err_badaddr;
default:
    return &err_nodev;
}

if (req[H_ERRNO]) {
    return sock_error(req[H_ERRNO]);
}
regs[0] = req[H_RESULT];
return 0;
```

Errors

A socket error goes back the way the Internet module sends it: an error block numbered &20E00 plus the `errno`, with the `errno`'s name as the message. The blocks are kept in a ring of four and used in turn, as the Internet module does. That protects a program that has not yet copied one error from a second error, raised by a callback as the SWI returns, overwriting it.

The module's own errors

Transport failures are not socket errors, so they use a different number, 0x1E4:

Message	When
"Internet: no HostNet device"	the doorbell was never found, or an unexpected transport code
"Internet: host serves no sockets"	HN_RC_NOSOCKETS: the device is there, sockets are off
"Internet: bad socket SWI"	HN_RC_BADSWI
"Internet: bad address"	HN_RC_BADADDR: a pointer in the call would not translate
"Internet: HostNet needs the emulator"	returned by initialisation, so the module does not load
"HostNet: no answer"	*HostNetPing got no magic number back

Refusing to exist

Sprint 0's module loaded even where there was no doorbell, and then refused every call. `3c523600ea` changed that, and its comment explains why a dormant module is worse than none:

```
/*
 * Refuse to start unless the doorbell is actually there.
 *
 * This matters more than the usual "stay dormant" politeness, because this
 * module is called Internet and claims the Internet module's SWI chunk.
 * A module that loads and then refuses every call does not sit quietly out
 * of the way -- it shadows the real Internet module, and the kernel gives
 * the later claimant of an in-use chunk priority (Kernel/s/ModHand). On a
 * real Raspberry Pi that would mean a machine with networking hardware,
 * a working stack in ROM, and nothing able to reach it.
 * ...
 */
```

hostnet_init

The initialisation that follows the comment maps the page and checks it twice:

```
int hostnet_init(void *ws)
{
    (void)ws;
    live = 0;
    cb_pending = 0;

    hn_base = os_map_io(HN_PHYS, HN_PAGE);
    if (!hn_base || hn_reg(HN_MAGIC / 4) != HN_MAGIC_VALUE) {
        /* No doorbell: this is not our emulator. Do not load. */
        return (int)&err_notemul;
    }
    if (!(hn_reg(HN_FEATURES / 4) & HN_FEATURE_SOCKETS)) {
        /* The emulator is ours but sockets were not asked for. Same
        * answer: better no Internet module than a deaf one. */
        return (int)&err_notemul;
    }
    live = 1;
    ...
    os_call_every(HN_TICK_CS, (uint32_t)hostnet_tick, static_base());
    ...
    os_service_internetstatus();
    return 0;
}
```

Returning an error block from initialisation makes the kernel refuse to load the module, which leaves SWI chunk &41200 unclaimed and a real Internet module, if there is one, in charge. The commit verified both ways. With the doorbell on, `*HostNetInfo` reported it live and `*GetHost` resolved. With it off, `*Modules` listed 138 modules, HostFS among them, no Internet module at all, and the machine still reached a clean desktop.

After registering the ticker, `hostnet_init` sends `Service_InternetStatus` with the `AddressChanged` reason, once, so that a program loaded later that waits for a working stack is not left waiting for ever. It does not send the `InterfaceUpDown` reason, which needs a description of an interface, and there is

no interface to describe.

`hostnet_final` removes the ticker before anything else. Its comment says why: "a ticker pointing into a module that is about to be unplugged is a branch into free memory."

Into the machine: two routes

Spliced into the ROM, or loaded from the disc

	ROM splice	Load from the disc
how	<code>rom.py</code> adds HostNet, ffa to the ROM image after HostFS, cached by content	!Boot's PreDesk.HostModules loads every module in HostFS:\$.Modules
the ROM's stack	Internet, EtherGENET, EtherUSB and DHCP unplugged in CMOS (98, 106, 107, 108)	still in the ROM; the kernel kills its Internet module as HostNet loads
the network card	none attached	attached, unused while HostNet runs
used by	<code>run.py --hostnet</code> , for development	the Windows release

Both routes keep MbufManager, chain 97, which looks pointless with no stack to buffer for. The `rom.py` comment gives the reason. !Internet's start-up checks for MbufManager and loads it from the disc if it is missing, and the only copy on the disc is a 26-bit module that cannot load on this machine. The start-up then stops with an error before running the file that sets the name servers. The machine reaches the desktop looking healthy, and NetSurf reports that no name servers are configured. Left in the ROM, MbufManager satisfies the check and allocates nothing.

Both routes share one hazard, which chapter 8 deals with. If HostNet loads while the doorbell is dark, the ROM's Internet module has already been unplugged or killed, HostNet declines, and the machine has no Internet module at all.

8. Shipping it

A working module and device are not yet something a user can switch on. This chapter covers the two ways HostNet reaches a running machine: the developer launcher's switch, which rebuilds the ROM, and the Windows release's switch, which is simply where a file sits on the disc. It walks through the release launcher, the HostNet item in the window menu, the one hazard both have to avoid, the installer's rules, and the start-up file that serves both network stacks. It ends with the Mac, where the work is not yet committed.

Two ways to reach a network

The machines and their stacks

Setup	Network card	ROM networking	Doorbell	Who serves sockets
<code>run.py</code> , the default	slirp and usb-net	running	off	the ROM's Internet 5.67
<code>run.py --hostnet</code>	none	unplugged: 98, 106, 107, 108	on	HostNet, spliced into the ROM
Windows release, HostNet on	attached, unused	Internet replaced as HostNet loads	on	HostNet, loaded from the disc
Windows release, HostNet off	slirp and usb-net	running	off	the ROM's Internet 5.67
Mac release and launchers	slirp and usb-net	running	off	the ROM's Internet 5.67

The developer's switch

`run.py`, the Windows developer launcher, takes `--hostnet`. The option splices HostNet into the ROM, unplugs the ROM's network modules in the CMOS it boots with, turns the doorbell on, and attaches no network card:

```
def command_line(args, overlay):
    # Two ways to reach a network, and never both. With --hostnet the
    # guest has no stack to drive a NIC with, so attaching one would be an
    # emulated card nothing ever opens; without it, slirp and usb-net as
    # before. The doorbell is off unless asked for, so a machine that has
    # not been switched over is untouched.
    net = (["-global", "hostnet.sockets=on"] if args.hostnet else
          ["-netdev", "user,id=n0,domainname=lan",
           # domainname: RISC OS asks for option 15 in its parameter
           # list. It does not fix DHCP, but it is one less thing the
           # guest asked for and did not get.
           "-device", "usb-net,netdev=n0,rndis=off,bus=usb-bus.0,port=1.3"])
```

This route rebuilds the ROM image and the CMOS blob with Python tools at launch. That suits a development machine and would not suit an end user's, which is why the release took the other route.

The Windows release: the module's folder

The Windows release keeps the ROM stock. HostNet ships as a module file on the disc, which is also the HostFS share, and the folder it is in is the switch:

The switch is a file

Where the file is	State	What happens at boot
Modules\HostNet,ffa	on	!Boot loads it; titled Internet, it replaces the ROM's Internet module
Modules\Disabled\HostNet,ffa	off	nothing loads it; the ROM's stack drives the network card

The loader is a one-line boot command file, `PreDesk.HostModules`, which the HostFS work added to `!Boot`:

```
IfThere HostFS:$.Modules Then Repeat RMLoad HostFS:$.Modules -Type &FFA -Sort -Continue
```

It loads every module file in `Modules`, in name order, and carries on past one that fails. It does not descend into subfolders, so a module in `Modules\Disabled` is never loaded. `make-release.py`'s `place_hostnet` puts the built module in one folder or the other, according to `--hostnet on` or `off`; `on` is the default.

The launcher

The Windows launcher is a small compiled program. For HostNet it does two things, described at length in its header comment:

- **It lights the doorbell only when HostNet is on.** Off is the device's default, and the machine then looks exactly as it did before HostNet.
- **It attaches the network card either way.** Under HostNet nothing uses the card. But if the user switches HostNet off and restarts RISC OS without restarting the emulator, the ROM's stack needs a card, or it boots with "Route: Network is unreachable".

The file test

The code is a file test:

```
/* The doorbell, lit only with HostNet on: its module in Modules itself.
 * The card is attached regardless, below; see the top of the file. */
_snwprintf(hostnet, MAX_PATH, L"%ls\\Modules\\HostNet,ffa", disc);
attrs = GetFileAttributesW(hostnet);
net = (attrs != INVALID_FILE_ATTRIBUTES
      && !(attrs & FILE_ATTRIBUTE_DIRECTORY))
      ? L"-global hostnet.sockets=on " : L"";
```

The hazard: a dark doorbell

Both switches share one way to leave a machine with no network stack at all, and the comment on the menu code spells it out. When a module titled Internet loads, the kernel kills the existing Internet module first, and only then runs the newcomer's initialisation. If the doorbell is dark at that moment, HostNet declines to load, as chapter 7 described, and there is no Internet module left.

What prevents it

Situation	Doorbell	What the design does
emulator started with HostNet on	lit by the launcher	nothing more needed
switched on from the menu, then RISC OS restarted	lit by the menu at the moment of switching	HostNet finds the doorbell and loads
switched off from the menu	left lit for this session	a HostNet already running keeps serving its sockets
file moved by hand to Modules mid-session	dark	the menu warns: on "when you quit and start again"

The window menu

The Direct3D window's system menu gains a **HostNet** item, rebuilt every time the menu opens. It appears only on a machine with a HostFS share whose `Modules` or `Modules\Disabled` folder holds the module, and it is ticked when the module is in `Modules`. Choosing it moves the file:

```
disabled = g_build_filename(dir, "Disabled", NULL);
from = g_build_filename(on ? disabled : dir, DX11_HOSTNET_LEAF, NULL);
to = g_build_filename(on ? dir : disabled, DX11_HOSTNET_LEAF, NULL);

if (!g_file_test(from, G_FILE_TEST_IS_REGULAR)) {
    g_snprintf(why, why_len, "%s is not there any more.", from);
    return -1;
}
if (g_mkdir_with_parents(on ? dir : disabled, 0755) != 0
    || (g_file_test(to, G_FILE_TEST_EXISTS) && g_remove(to) != 0)
    || g_rename(from, to) != 0) {
    g_snprintf(why, why_len, "Cannot move %s to %s: %s", from, to,
               g_strerror(errno));
    return -1;
}
if (on) {
    bql_lock();
    hn->enabled = true;
    bql_unlock();
}
return 0;
```

The paths are built with GLib from the share's root, as HostFS builds them, so the menu and the filing system agree about where `Modules` is. Switching on also lights the doorbell, under QEMU's global lock. Nothing ever darkens it.

The menu's notes

The menu knows three things: where the file is, whether the doorbell is lit, and whether the running RISC OS has rung it since it started. When the tick and the running machine disagree, a greyed note says so:

File in	Lit	Rung	Note under the item
Modules	yes	yes	none: on, and running
Modules	yes	no	"On when RISC OS next starts"
Modules	no	no	"On when you quit and start again"
Disabled	any	yes	"Off when RISC OS next starts"
Disabled	any	no	none: off

The "rung" flag is the device's `rung` field from chapter 3. The module rings fifty times a second, so within one tick of HostNet loading the menu knows that HostNet is what this boot is running, even after the file has been moved.

The installer

The Windows installer, built with Inno Setup, installs the disc into the user's profile folder, and never overwrites a file already there. HostNet is the exception, with its own entry and two rules, written in the installer's Pascal script:

```
{ Where HostNet's module goes: where it already is, or for a new machine
  where the release starts it, @HOSTNET_DIR@. }
function HostNetDir(Param: String): String;
begin
  if FileExists(DiscDir() + '\Modules\HostNet,ffa') then
    Result := DiscDir() + '\Modules'
  else if FileExists(DiscDir() + '\Modules\Disabled\HostNet,ffa') then
    Result := DiscDir() + '\Modules\Disabled'
  else
    Result := DiscDir() + '@HOSTNET_DIR@';
end;
```

The two rules

Rule	Why
the module is always replaced, in whichever folder the user left it	the module and the emulator it talks to are one version, and a reinstall must never flip the switch
a disc from before HostNet does not get the module at all	that disc's start-up file was written for the network card alone, and fails under HostNet before the name servers are set

The second rule is `HostNetWanted`: a new machine gets the module, and so does one that already has it in either folder. An older machine gets neither the module nor, therefore, the menu item that would switch it on.

One start-up file for two stacks

The disc's `Choices:Internet.Startup` runs whichever stack is the Internet module. The network card needs interfaces, an address and a route configured; under HostNet those commands fail, because there is no interface, and the `CheckError` after them would stop `!Internet`'s start-up before the file that sets the name servers.

The shared start-up file

`e8bdae8d8d` therefore splits the file in two. `make-release.py` writes a `Startup` that holds only what both stacks need, and hands the card's configuration to a second file when the stack is the ROM's:

```
Set Inet$HostName RISCOSpi

| Read by !Run after this file returns: no gateway, no RouteD.
Set Inet$IsGateway ""
Set Inet$RouteDOptions ""

| The interfaces, only when the Internet module is older than HostNet.
RMEnsure Internet 6.00 Run Choices:Internet.Interfaces
```

`RMEnsure` runs its command when the named module is older than the version given. HostNet is Internet 6.00, so under HostNet the interfaces file is skipped; the ROM's Internet 5.67 is older, so with HostNet off it runs. `Choices:Internet.Interfaces` holds the card's configuration: the fixed slirp address from `d0de1c013c`, the route, loopback and the file-sharing modules. The name servers stay in the `User` file, which both stacks run.

A banner at the top of `Startup` warns that saving from `!InetSetup` replaces the file with one that "knows nothing of HostNet".

What the public site tells users

The fork's public site describes the Windows release in plain terms. Paraphrased: HostNet redirects RISC OS's IP sockets to Windows, which makes the connections, and it should be faster and more reliable than the emulated USB network. It is not a complete network stack. It serves TCP and UDP

programs, such as browsing, name lookups, fetching files and programs that listen, but tools like `Ping`, `IfConfig`, `ARP`, `route` and `InetStat` have nothing to report, and `ShareFS`, `Access` and `Econet` do not work over it. It is IPv4 only, on at install, switched from the window menu, and a switch takes effect when RISC OS next starts.

The Mac

At `d535505fad`, no committed Mac launcher knows about HostNet. The Mac app and the Mac development scripts still attach `slirp` and `usb-net`, and the only committed Mac change is the device build fix, `20bf267db5`. A complete Mac integration existed in the working tree on 15 September, uncommitted, and follows a different model from the Windows release: the ROM splice and CMOS unplug, with no card. Chapter 9 describes it.

9. Proven, and what comes next

This chapter gathers what the commits prove, with the numbers, then the limits the project states for users, and finally the work that was in progress on 15 September.

What each step proved

Every sprint commit closes with a test run on the emulator:

The tests

Step	Test	Result
Sprint 0, 23fc9630c5	boot with no network card; *Modules; *HostNetPing; start NetSurf	desktop reached; Internet at the spliced address, EtherUSB, EtherGENET and DHCP gone; ping answered; NetSurf idle with no error
Sprint 1, 0f73da70af	*GetHost www.riscosopen.org through the stock Resolver	91.203.57.12, in one exchange: a 30-byte query, a poll, a 94-byte answer
Sprint 1, failure path	the resolver pointed at an unreachable address	eight sends, then "Failed to look up", with the machine still running
Sprint 2, 89013ef0f2	NetSurf over TLS	riscosopen.org in 1.3 s; Wikipedia's RISC OS article in 5.3 s over 27 sockets

The tests, continued

Step	Test	Result
Sprint 3, 5b882f2a4a	listen.bas, a guest server	18 bytes echoed; seven retries in the blocking accept
Sprint 3	evtest.bas, events only	async, then broken; eight bytes read on the event
50ef1a1bcd	registers, off and on	all zero off; 'HNET', 1, 1 on
3c523600ea	boot with the doorbell off	138 modules, no Internet module, a clean desktop
e8bdae8d8d	a staged Windows release	both states boot and resolve; the menu moves the file

The numbers

Measurement	Value	Source
DHCP success on the old stack	0 of 12 boots	23fc9630c5
USB transactions per 1500-byte frame, old path	24	23fc9630c5
desktop time, DHCP against static	49 s against 28 s	d0de1c013c
NetSurf, riscosopen.org over TLS	1.3 s; the old stack failed	89013ef0f2
NetSurf, Wikipedia's RISC OS article	5.3 s, 27 sockets	89013ef0f2

No throughput or latency figure exists for HostNet yet. Measuring throughput against the old path is part of Sprint 5, which has not started.

What HostNet does not do

These are the limits the code and the public notes state:

Limit	Where stated
IPv4 only; AF_INET6 answers EAFNOSUPPORT	hn_creat; the public site
no raw sockets or ICMP, so Ping, IfConfig, ARP, route and InetStat have nothing to report	the public site
ShareFS, Access and Econet do not work over it	the public site
a blocking call outside a TaskWindow holds the desktop until it completes or Escape is pressed	the module's wait loop comment
a restored snapshot has no open sockets	the saved-state comment
saving from !InetSetup replaces the start-up file that serves both stacks	the banner in Startup

Work in progress

The local checkout on 15 September held uncommitted work that brings HostNet to the Mac, based on 3c523600ea; this describes it as it stood. build-hostnet.sh finds its tools on either host. RISCOS_HOSTNET=1 makes the Mac scripts splice HostNet, unplug the ROM stack and attach no card, and the Mac app ships it on by default, with a small shell script that converts an older disc at launch. The Mac notes record a run on 14 September: *GetHost resolving, NetSurf loading Wikipedia's RISC OS article in 3.7 seconds, both BASIC tests passing, and 13 guest sockets counted by both the trace and lsof.